Assignment - 08

Problem Statement
Write a C program to evaluate postfix expression using stack

Objective:
1. To study Stack and its operations
2. To study the importance of expression evaluation

Theory:
Expression evaluation is the process of calculating the value of an arithmetic expression. A stack is the most suitable data structure because it follows the Last In First Out (LIFO) principle, which matches the order in which operators and operands are processed.
Expression evaluation is commonly performed on:
- Postfix (Reverse Polish Notation)
- Prefix (Polish Notation)

Infix expressions are usually converted to postfix or prefix before evaluation.

Example:
Evaluate the expression 3 5 + 4 *

Symbol	Operation	Stack
3	Push 3	3
5	Push 5	5, 3
+	Pop 5, pop 3, perform 3+5, push 8	8
4	Push 4	4, 8
*	Pop 4, pop 8 perform 4*8 push 32	32

Implementation:
- Platform
  - 64-bit Open source Linux or its derivatives
  - Open Source C Programming tool like gcc/Eclipse Editor
- Test Conditions:
  1. Input postfix expression
  2. Evaluate the value of expression.

Pseudo Code:
1. Initilize an empty stack
2. Scan the postfix expression from left to right.
3. For each token:
   a. If operand, push onto the stack.
   b. If operator, pop two operands (op2, op1), apply operator, and push result
4. Return the final element remaining on the stack.

Time Complexity

Time Complexity: O(n) - Every token scanned.
Space Complexity: O(n)

Conclusion
Thus, implemented Stack Operations to evaluate the value of an expression.

FAQs

1) Explain how postfix/prefix expression is evaluated:
Postfix: Scanned left to right; operands are pushed, and operators pop operands to compute and push results back.
Prefix: Scanned right to left using a similar stack based mechanism

2 What is ISP and ICP?

- ISP (In-Stack Priority): Precedence of an operator already inside the stack.
- ICP (Incoming Priority): Precedence of an incoming operator being evaluated. Both determine when to push or pop during expression conversion

3 Give various applications of stack.

- Expression evaluation and conversion
- function calls and recursion
- Parenthesis matching
- Backtracking algorithms (DFS, mazes)
- ~~Undo~~ Undo/redo mechanisms in software.

5/10